

PROJECT SPECIFICATION

Sentinel

AI-Powered Error Observability SDK for FastAPI

Drop-in middleware that captures request logs and tracebacks, embeds them with the Gemini API, and lets developers ask a built-in `/sentinel` dashboard why an error happened and where to fix it — powered by Retrieval-Augmented Generation.

FastAPI Middleware

Gemini Embeddings

RAG Search

`/sentinel` Dashboard

Zero-config SDK

1. Overview

Sentinel is a lightweight Python SDK that plugs into any FastAPI application as middleware. It transparently records request/response metadata and exceptions, then exposes a built-in web dashboard at `/sentinel` once the server is running — locally or in production. The dashboard doesn't just list raw logs: every captured error is embedded using the **Gemini API** and stored in a lightweight vector index. When a developer asks a question like “why is this 500 happening?”, Sentinel runs a **Retrieval-Augmented Generation (RAG)** query over past logs, stack traces, and related request context, then asks Gemini to explain the likely root cause and point to the offending file/line.

Goal: turn “scroll through terminal logs” debugging into “ask a question, get an answer with evidence” debugging — without shipping logs to a third-party SaaS.

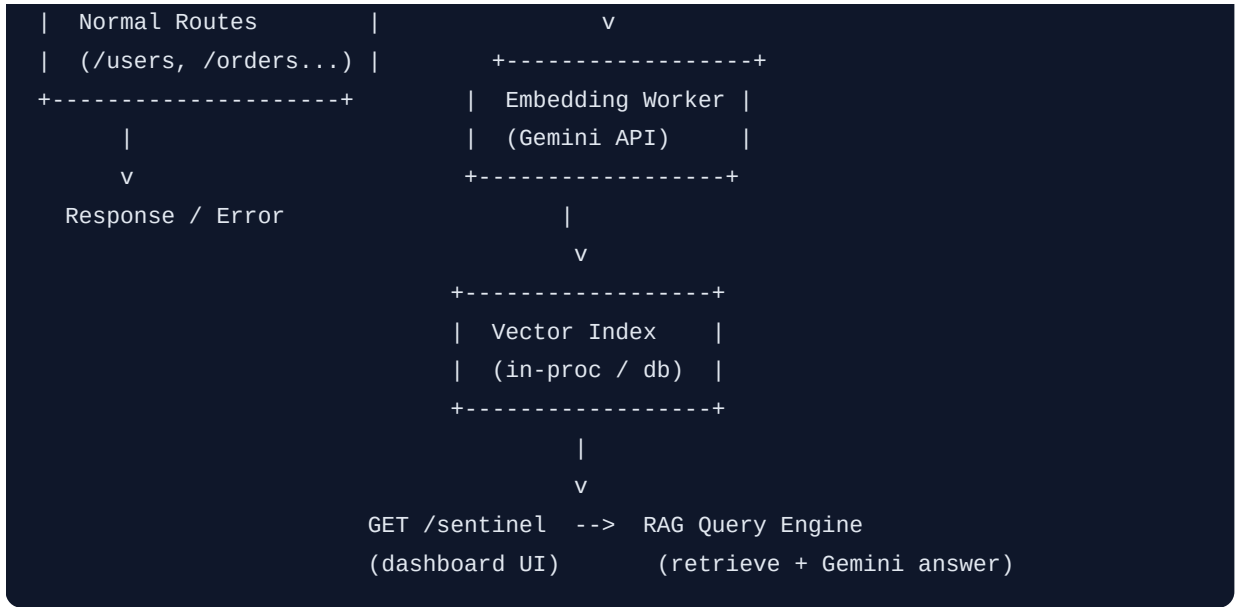
1.1 Core Value Proposition

- **Zero-config install** — one line of middleware registration, no external log aggregator required.
- **Self-hosted dashboard** — `/sentinel` is served directly by the same FastAPI app, so it works identically on localhost and in production.
- **Semantic error search** — instead of grepping logs, ask in plain English; Gemini embeddings find semantically related past errors, not just string matches.
- **Root-cause explanations** — RAG grounds Gemini's answer in the actual traceback, request payload, and code context, reducing hallucinated fixes.
- **Pluggable storage** — SQLite by default for local dev, swappable for Postgres/pgvector or Redis in production.

2. System Architecture

Sentinel has four moving parts: the **middleware** that intercepts every request, the **log store** that persists structured events, the **embedding + RAG engine** that indexes and retrieves errors via the Gemini API, and the **`/sentinel` dashboard** that serves the UI and chat-style query endpoint from within the same FastAPI process.





3. Request/Error Middleware

The core of the SDK is a single ASGI middleware class registered via `app.add_middleware(SentinelMiddleware)`. It wraps every request in a `try/except`, capturing timing, status code, and — on failure — the full exception with traceback, local variables (optionally redacted), and the originating route.

3.1 What gets captured per event

Field	Description
request_id	UUID correlating request, response and any error
method / path	HTTP method and route path (e.g. POST /api/orders)
status_code	Final response status
duration_ms	Request latency
exception_type	e.g. KeyError, ValidationError, IntegrityError
traceback	Full formatted stack trace
source_location	File path + line number where the exception originated
request_meta	Headers (sanitized), query params, path params
body_snippet	Truncated, PII-redacted request body for context

Field	Description
embedding_id	Pointer to the Gemini embedding vector for this event

3.2 Example integration

```
from fastapi import FastAPI
from sentinel_sdk import SentinelMiddleware, SentinelConfig

app = FastAPI()

app.add_middleware(
    SentinelMiddleware,
    config=SentinelConfig(
        gemini_api_key="env:GEMINI_API_KEY",
        storage_url="sqlite:///./sentinel.db",
        dashboard_path="/sentinel",
        redact_fields=["password", "authorization", "token"],
        capture_request_body=True,
    ),
)
```

4. Embeddings & RAG Pipeline

Every captured error event is turned into a short structured document (exception type, message, traceback, route, and a redacted request snippet), embedded with the Gemini embedding model, and stored alongside the raw event. When the user asks a question in the dashboard, Sentinel embeds the question, retrieves the top-k most similar past events plus the current session's recent logs, and passes that context to Gemini's generation endpoint to produce a grounded explanation.

4.1 Indexing flow

- Error occurs → middleware builds a structured **ErrorDocument** (type, message, trace, route, request context).
- Document text is sent to the **Gemini embedding model** to produce a vector.
- Vector + metadata are written to the vector index (in-memory for dev, pgvector/FAISS for scale).
- Document is also stored in the relational log table for the raw dashboard view.

4.2 Query (RAG) flow

- Developer opens /sentinel and asks: "Why does POST /checkout keep returning 500?"
- Question is embedded and matched against the vector index (cosine similarity, top-k=5–10).
- Matched events + their tracebacks are assembled into a context window.
- Context + question are sent to Gemini's chat/generation endpoint with a system prompt instructing it to cite the specific file/line and explain root cause, not just repeat the traceback.
- Answer streams back to the dashboard with clickable links to the exact log entries used as evidence.

4.3 Example: embedding + retrieval

```
# sentinel_sdk/rag/engine.py

async def index_error(event: ErrorEvent) -> None:
    doc_text = build_error_document(event)
    vector = await gemini_embed(doc_text, model="gemini-embedding-001")
    await vector_store.upsert(id=event.id, vector=vector, meta=event.to_meta())

async def answer_question(question: str, session_id: str) -> RagAnswer:
    q_vector = await gemini_embed(question, model="gemini-embedding-001")
    matches = await vector_store.query(q_vector, top_k=8, session_id=session_id)
    context = build_context_block(matches)

    prompt = SENTINEL_SYSTEM_PROMPT.format(context=context, question=question)
    response = await gemini_generate(prompt, model="gemini-2.5-flash")
```

```
return RagAnswer(text=response.text, sources=[m.id for m in matches])
```

5. /sentinel Dashboard

The dashboard is mounted as a sub-application at `/sentinel`, served by the same FastAPI process — no separate deployment needed. It has three views:

5.1 Views

- **Live Log Stream** — real-time feed of requests and errors via WebSocket, filterable by route, status code, and time range.
- **Error Explorer** — grouped/deduplicated errors (by exception type + route) with frequency counts and trend sparklines.
- **Ask Sentinel (RAG chat)** — a chat box where the developer types a question and gets a grounded, cited answer with links back to the specific log entries and source locations.

5.2 API surface (mounted under /sentinel)

Endpoint	Method	Purpose
<code>/sentinel</code>	GET	Dashboard UI (HTML/JS)
<code>/sentinel/api/logs</code>	GET	Paginated, filterable log query
<code>/sentinel/api/logs/stream</code>	WS	Live log stream
<code>/sentinel/api/errors/grouped</code>	GET	Deduplicated error groups + counts
<code>/sentinel/api/ask</code>	POST	RAG question → grounded answer + sources
<code>/sentinel/api/health</code>	GET	SDK + storage + Gemini connectivity check

6. Tech Stack

Layer	Choice	Notes
Middleware	Starlette BaseHTTPMiddleware / pure ASGI	Matches FastAPI's underlying framework
Storage (dev)	SQLite (aiosqlite)	Zero-config local persistence
Storage (prod)	PostgreSQL + pgvector	Scales log volume and vector search together
Embeddings	Gemini Embedding API	Configurable model, batched calls
Generation	Gemini API (chat/generate)	Streams grounded answers for the dashboard
Vector search	pgvector or in-process FAISS	Chosen based on storage backend
Dashboard frontend	Server-rendered + lightweight JS (HTMX or React build)	Ships as static assets bundled with the package
Packaging	PyPI package (pip install sentinel-sdk)	Optional extras: sentinel-sdk[postgres]

7. Security & Privacy Considerations

- **Field redaction** — configurable deny-list (passwords, tokens, auth headers) stripped before storage or embedding.
- **Local-first by default** — SQLite + local vector index; nothing leaves the server except the text sent to the Gemini API for embeddings/generation.
- **Dashboard auth** — `/sentinel` should be protected (basic auth, IP allowlist, or app's existing auth dependency) since it exposes stack traces and request bodies.
- **API key handling** — Gemini API key read from environment/secret manager, never logged or embedded in stored documents.
- **Data retention** — configurable TTL to auto-purge old log/vector entries.

8. Suggested Build Roadmap

Phase	Deliverable
1. Core middleware	Capture request/error events, write to SQLite
2. Dashboard v1	/sentinel with live log stream + error explorer (no AI yet)
3. Gemini embeddings	Index every error event as a vector on capture
4. RAG chat	/sentinel/api/ask endpoint + chat UI, grounded answers with citations
5. Production hardening	Postgres/pgvector backend, auth, redaction, retention policy
6. Packaging & docs	Publish to PyPI, quick-start docs, example FastAPI repo

This document is a technical starting point — endpoint names, config options, and storage choices can be adjusted as the SDK's real-world usage patterns emerge.